操作系统面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

什么是用户态和内核态？

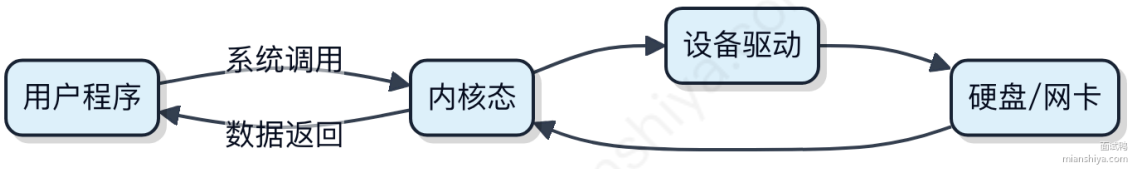
操作系统为了保护系统资源，会把运行环境划分为用户态和内核态。

用户态下，进程只能访问受限的内存区域，不能直接操作硬件。我们写的 Java 程序、Nginx 服务，跑的都是用户态。这时候如果需要读写文件、发送网络请求，就得通过系统调用陷入内核态。

内核态拥有最高权限，能访问所有内存和硬件设备。像磁盘 I/O、网络收发包、进程调度这些脏活累活都在这里完成。比如一次 `read()` 调用，用户态程序发起请求后，CPU 切到内核态执行驱动代码，数据从网卡拷贝到内核缓冲区，再复制到用户内存。

这个切换过程是有成本的。上下文切换、特权级转换都会消耗 CPU 周期。所以高性能服务要尽量减少系统调用次数，像 Netty 用零拷贝、epoll 边缘触发就是为了压低陷入内核的频率。

反过来，如果所有操作都在用户态搞定，系统就不受控了，随便一个程序都能干掉整个机器。所以这种分层是必须的。



简单说，用户态负责业务逻辑，内核态负责资源管理和硬件交互，两者通过系统调用桥接。

什么是 CPU 缓存一致性协议 MESI？

CPU 缓存一致性是多核编程里绕不开的底层机制。MESI 其实就是四个状态的缩写，用来保证多个 CPU 核心的缓存数据是一致的。

每个缓存行（cache line）在任意时刻处于以下四种状态之一：

1）**Modified**：这行数据被当前核心修改过，和内存不一致，只有本核心有最新值。

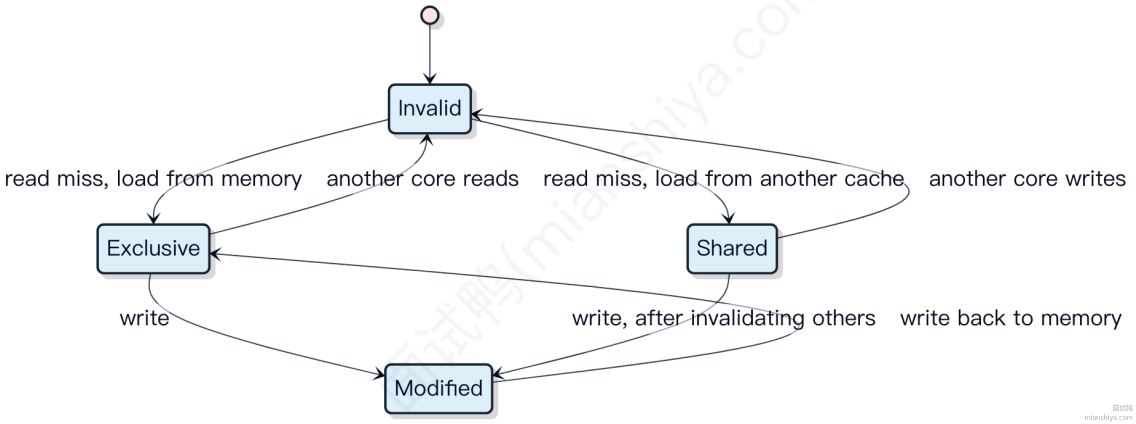
2）Exclusive：数据和内存一致，但只有当前核心在用，没人共享。

3）Shared：多个核心都在读这个数据，大家缓存的值都一样。

4）Invalid：这行缓存无效，不能直接用，得先去加载。

当某个核心要写数据时，会触发总线嗅探（bus snooping），其他核心收到通知后检查自己的缓存，如果也有这行数据，就把它置为 Invalid。这样下次访问就得重新从内存或其它核心拉取最新值，避免脏读。

比如你在 Java 里写 volatile 变量，背后就会触发 MESI 协议来确保可见性。虽然 volatile 不保证原子性，但它能强制刷新缓存，让修改立刻被其他核心看到。



简单说，MESI 就靠这套状态机 + 总线广播，让多核之间的缓存看起来像是“同步”的。

负数的二进制如何表示？

计算机里负数是用补码表示的，不是简单把符号位改成1就完事。你得明白原码、反码、补码这套逻辑。

正数的二进制就是它本身的原码，比如 5 在 8 位系统里是 `0000 0101` 。那 -5 怎么搞？先看原码：符号位为 1，后面接 5 的二进制，得到 `1000 0101` 。但这只是原码，不能直接用来运算。

反码是把原码的符号位不变，其余位取反，-5 的反码就是 `1111 1010` 。还是不行，因为 0 会有两个表示（+0 和 -0），计算也不方便。

补码才是关键。负数的补码 = 反码 + 1。所以 -5 的补码是 `1111 1011` 。这样一来，加减法就能统一处理了，CPU 做 `5 + (-5)` 直接按二进制相加，结果是 `1 0000 0000` ，溢出后变成 `0000 0000` ，刚好是 0。

还有一个更简单的记法：从右往左，第一个 1 出现之前的所有位保持不变，包括这个 1，之后的每一位取反。比如 5 是 `0000 0101` ，从右开始第一个 1 是第三位，前面都保留，后面取反，符号位置 1，也能得出 `1111 1011` 。

Java 里 `Integer.toBinaryString(-5)` 输出的是无符号形式，会显示一长串 1 开头的，其实是补码的完整表达。

操作系统中的进程有哪几种状态？

进程在操作系统里不是一直跑着的，它会因为资源、调度或自身行为在不同状态间切换。常见的状态有五种，这是理解系统调度和程序执行的关键。

1）新建（New）
进程刚被创建出来，还没进就绪队列。这时候操作系统正在给它分配 PCB（进程控制块），准备资源。

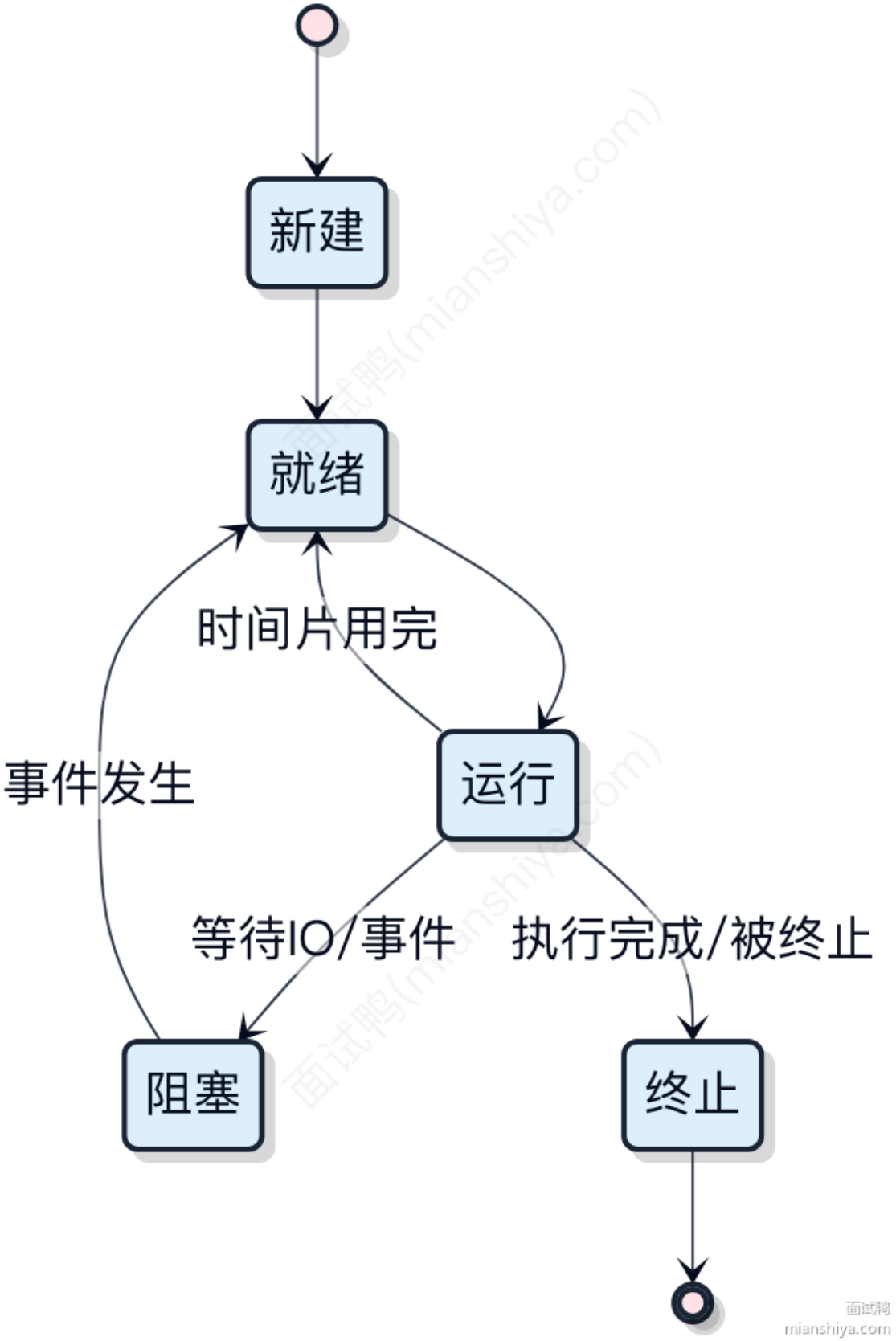
2）就绪（Ready）
进程已经准备好，就等着 CPU 分了。它在就绪队列里排队，只要调度器选中它，立马就能运行。

3）运行（Running）
当前正在 CPU 上执行的进程。单核情况下，同一时间只有一个进程处于这个状态。

4）阻塞/等待（Blocked/Waiting）
进程想干点事但条件不满足，比如等磁盘 IO、等网络数据、等锁释放。这时候它不参与调度，**CPU 压根不经过它**。直到事件发生，比如数据到了，才回到就绪态。

5）终止（Terminated）
进程执行完了，或者被 kill 了。操作系统开始回收它的资源，比如内存、文件描述符。

面试鸭(mianshiya.com)
面试鸭(mianshiya.com)


新建
就绪
时间片用完
事件发生
运行
等待IO/事件
执行完成/被终止
阻塞
终止
面试鸭
mianshiya.com

注意，有些教材会把“挂起”状态单独列出来，那是为了处理内存不足时的 swapped out 情况。但基础模型还是这五个状态打底。

I/O是什么？

I/O 其实就是数据在不同设备之间的搬运工，比如从磁盘读文件、往网络发数据。操作系统把所有外部设备都抽象成“文件”，所以这类操作统称 I/O。

1）按阻塞方式分，有**阻塞 I/O** 和非阻塞 I/O。最常见的 read/write 调用就是阻塞的，程序会卡住等数据准备好。像 Tomcat 处理 HTTP 请求时，如果用传统 BIO，每个连接都要一个线程盯着，上千并发就得上千线程，系统压根扛不住。

2）为了提升效率，就出了 NIO。Java 里的 `Selector` 能监听多个通道的状态变化，一个线程就能处理几百个连接。Netty 就是基于这个模型，用少量线程撑起高并发通信。

3）还有 mmap、sendfile 这类零拷贝技术，让数据不用来回拷贝进用户内存。比如 Kafka 发消息，直接通过 sendfile 把磁盘文件推到网卡，性能一下子提上来。

注意传统 I/O 至少要两次拷贝和四次上下文切换，开销不小。现在搞高性能网关或者消息中间件，底层 I/O 模型选型特别关键，直接决定能扛多少流量。

说下你常用的 Linux 命令？

排查问题常用的命令，其实就围绕着看日志、看资源、抓进程、网络状态这几个方向。

查日志肯定用 `tail -f` 实时跟踪应用输出，比如：

```
tail -f /var/log/app.log
```

配合 `grep` 搜关键字，加个 `--color` 看得清楚：

```
grep --color "ERROR" /var/log/app.log
```

看系统负载，`top` 或更顺手的 `htop` 一眼就能看到 CPU、内存占用高的进程。内存吃紧时特别关注 `RES` 和 `%MEM` 这两列。

找进程靠 `ps` 搭配 `grep`：

```
ps aux | grep java
```

杀进程先 `kill -15` 让程序优雅退出，搞不定再 `kill -9` 强杀。

网络方面，`netstat -tunlp` 或 `ss -tulnp` 查端口占用，比如发现 8080 被谁占了：

```
ss -tulnp | grep 8080
```

连通性用 `curl` 测接口最直接，比如调下游服务：

```
curl -v http://api.example.com/health
```

面试鸭(mianshiya.com)

大文件传输时用 `scp` 或 `rsync` 同步目录也常用：

```
rsync -avz ./logs/ user@192.168.1.10:/backup/
```

临时要看磁盘空间，`df -h` 看整体，`du -sh *` 定位具体目录大小。

这些命令组合起来，基本能扛住日常运维和线上问题排查的脏活累活。

什么是软中断、什么是硬中断？

中断是操作系统响应外部事件的核心机制，分硬中断和软中断，它们分工明确。

硬中断由硬件设备触发，比如网卡收到数据包、键盘按下按键。CPU 接收到信号后会立刻暂停当前任务，跳转到对应的中断处理程序。这个过程压根不经过用户代码，由硬件直接通知 CPU，所以必须快。硬中断处理函数要短小精悍，一般只做标记工作，比如设置一个标志位，告诉系统“有事来了”。

真正干活的是软中断。它在内核态中被触发，通常由硬中断唤醒。比如网卡中断来了，硬中断只是标记“有包到了”，然后触发一个 NET_RX 软中断。后续的数据包收取、协议栈处理都在软中断上下文中完成。这样能避免硬中断占着 CPU 太久，影响系统响应。

Linux 用 `ksoftirqd` 内核线程来兜底执行软中断，防止积压。如果软中断处理不过来，系统吞吐就会下降，常见于高网络吞吐场景，比如用 **Nginx** 或 **Kafka** 时网卡收包量大，软中断 CPU 占用率可能飙到 30% 以上。

1）硬中断是硬件发信号，响应必须快
2）软中断是内核里延后处理的“任务队列”
3）两者协作，才能既及时又高效地处理外部事件

进程之间的通信方式有哪些？

进程间通信本质是数据交换，不同场景下选择合适的方式能避免资源浪费或性能瓶颈。

1）管道是最简单的单向通信方式，父子进程间用匿名管道就行，比如 shell 命令 `ps | grep java` 就是通过管道传递数据。有名管道（FIFO）则允许无亲缘关系的进程通过文件路径关联读写端。

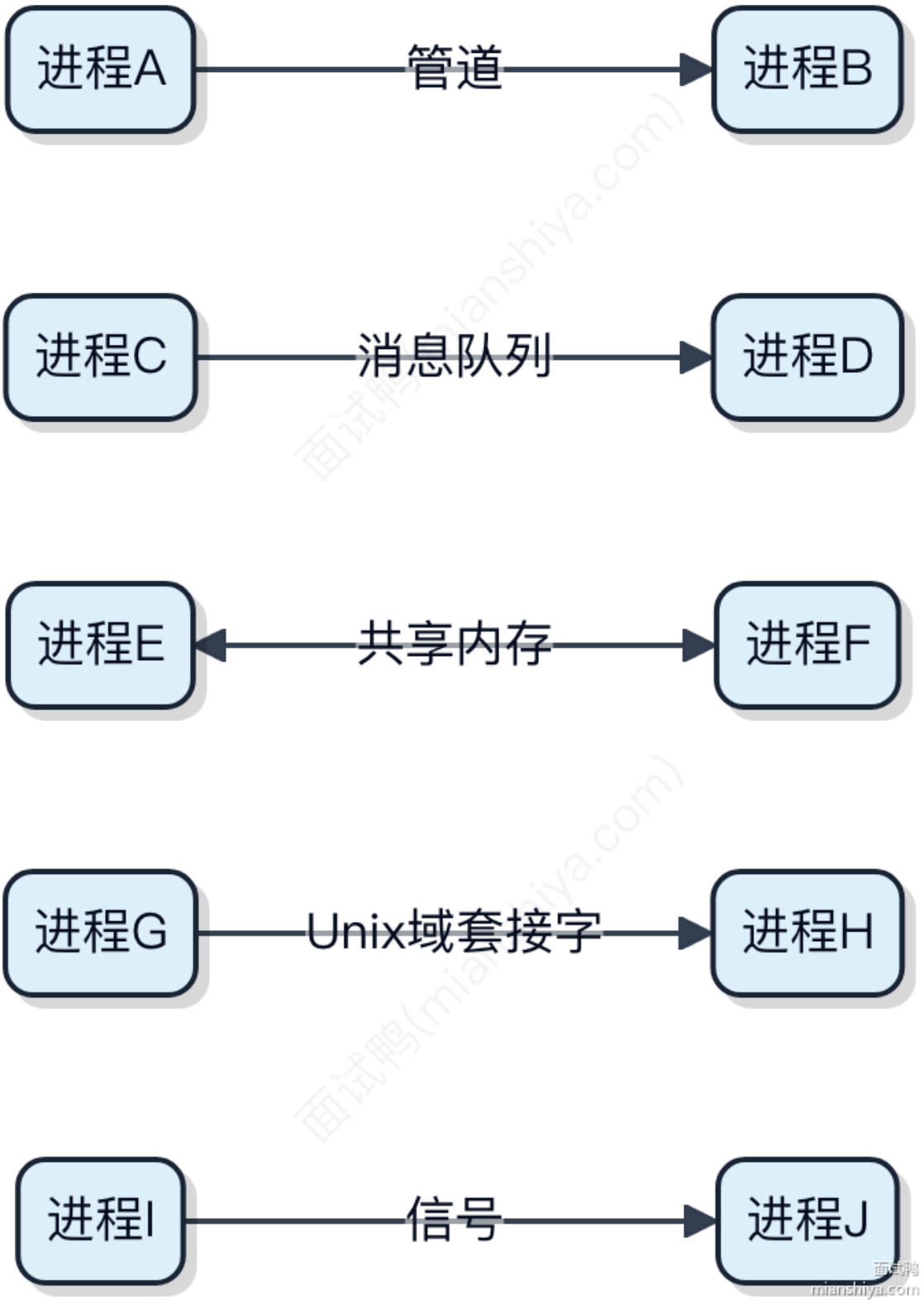
2）消息队列适合异步解耦，像 RocketMQ 或 RabbitMQ 这类中间件底层就用了类似机制，不过系统级的消息队列如 POSIX 消息队列可以直接在本地进程间传递结构化消息，带优先级和类型过滤。

3）共享内存效率最高，多个进程映射同一块物理内存，通信不经过内核拷贝。但需要额外同步手段，比如配合信号量控制访问顺序，否则容易出错。数据库连接池这类高频访问场景会用到。

4）信号属于低开销通知机制，只能传递简单事件，比如 kill -9 发送 SIGKILL 终止进程，不能传数据。

5）套接字不限于网络，本地可以用 Unix 域套接字（AF_UNIX），像 Nginx 和 PHP-FPM 之间就常用这种方式通信，传输效率比 TCP 高，压根不经过网络协议栈。

6）信号量严格来说不算传数据，而是协调资源访问，防止竞争，常用于控制对共享内存的访问权限。


进程A
管道
进程B
进程C
消息队列
进程D
进程E
共享内存
进程F
进程G
Unix域套接字
进程H
进程I
信号
进程J

[线程和进程有什么区别？](#)

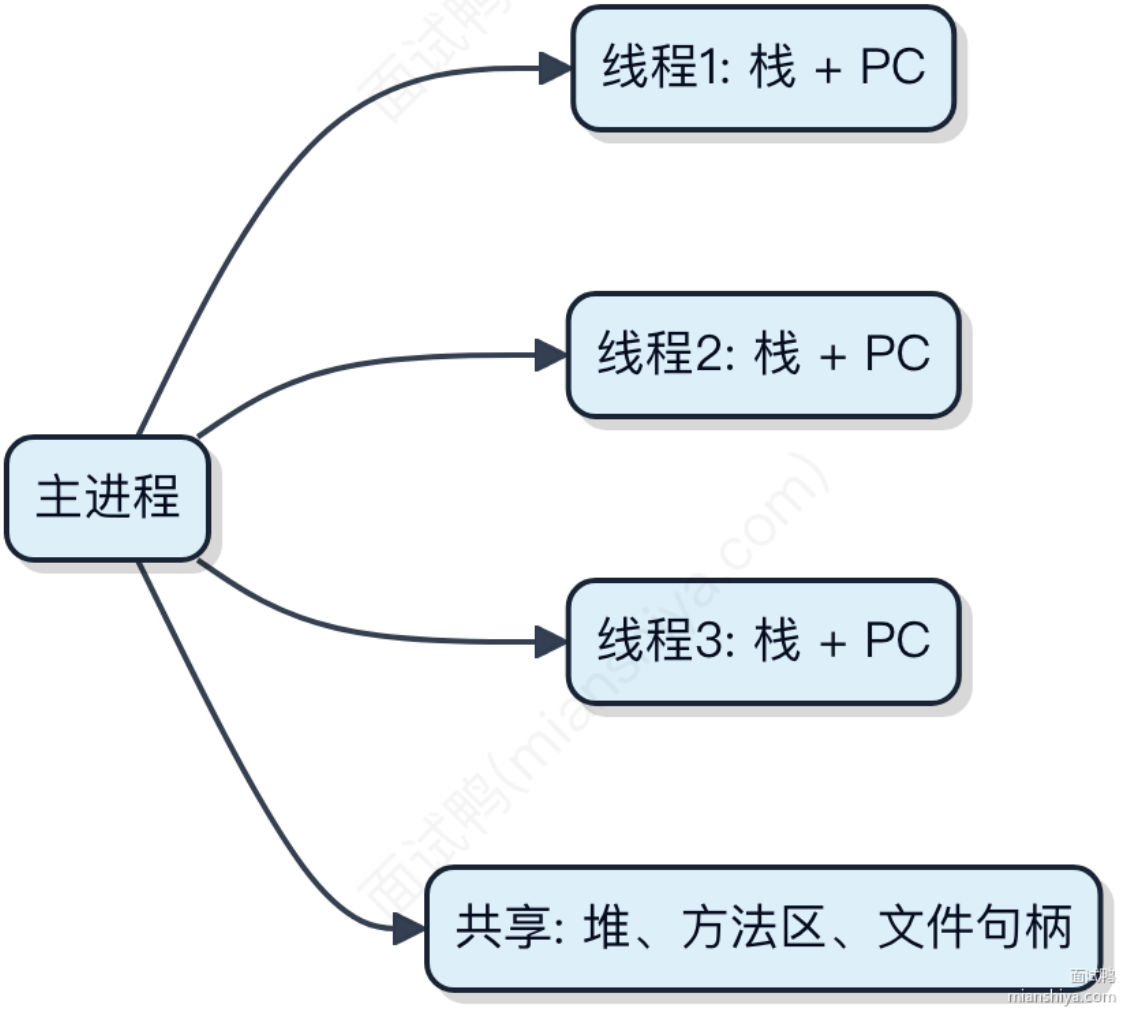
进程是资源分配的基本单位，线程是CPU调度的基本单位。一个进程可以包含多个线程，这些线程共享进程的内存空间和文件描述符等资源，但每个线程有自己独立的程序计数器、栈和寄存器状态。

1）进程之间相互隔离，一个进程崩溃通常不会直接影响另一个进程。比如你运行两个Java应用，它们分别在不同的JVM进程中，一个OOM了，另一个照常运行。

2）线程共享所属进程的堆内存，所以多线程编程要注意数据竞争问题。像HashMap这种非线程安全的结构，多个线程同时写就会出问题，得用ConcurrentHashMap或者加锁来保护。

3）创建进程开销大，因为要分配独立的地址空间，还要做页表映射。Linux下fork()虽然用了写时复制优化，但依旧比创建线程贵得多。Java里new Thread()底层对应pthread_create，在同一个进程内启动新执行流，代价小很多。

4）线程上下文切换成本低于进程。进程切换时CPU要把整个地址空间的映射都换掉，而线程切换只刷新少量寄存器，TLB也可能不用全清。



多线程适合高并发场景，比如Tomcat用线程池处理HTTP请求，每个请求一个线程；而多进程更适合隔离性要求高的场合，比如浏览器为每个标签页启动独立进程防崩溃。

什么是分段、什么是分页？

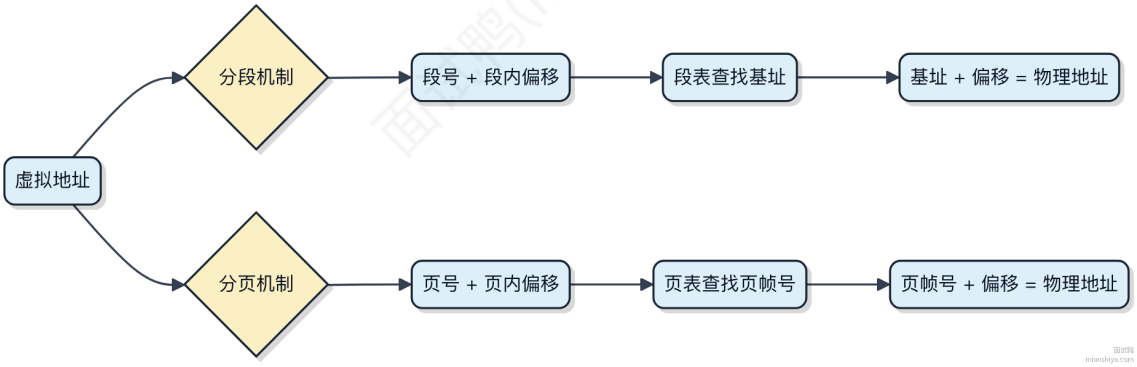
分段和分页是操作系统管理虚拟内存的两种机制，解决程序地址空间如何映射到物理内存的问题。

分段把程序按逻辑模块划分，比如代码段、数据段、堆栈段。每个段有独立的基地址和长度，通过段表做映射。这样做的好处是便于实现共享和保护，比如多个进程可以共用同一个代码段。但容易产生外部碎片，内存利用率低，因为段大小不一，回收后留下零散空隙。

分页则把虚拟和物理内存都切成固定大小的块，虚拟内存叫页，物理内存叫页帧，一般大小是 4KB。通过页表把虚拟页号映射到物理页帧号。由于块大小统一，分配和回收不会产生外部碎片，只有少量内部碎片（最后一页用不满）。但页对用户不可见，不利于程序的模块化管理和共享。

举个例子，一个程序要访问虚拟地址，先拆解出页号和页内偏移，查页表得到对应页帧号，拼接偏移量就得到物理地址。这个过程由 MMU 硬件自动完成，多数情况下还能借助 TLB 缓存加速。

两者根本区别在于划分方式：分段是逻辑驱动的变长划分，分页是硬件驱动的等长划分。现代系统通常采用段页式结合，先分段再分页，兼顾逻辑隔离和内存效率。



操作系统在进行线程切换时需要进行哪些动作？

线程切换本质是把 CPU 的执行权从一个线程转移到另一个，这个过程由操作系统内核调度器触发，比如时间片用完、主动让出或阻塞等待。

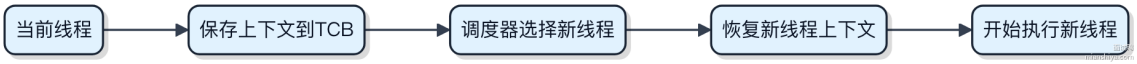
1）首先会保存当前线程的**上下文**。这包括程序计数器（PC）、寄存器状态、栈指针等，这些数据会被存到该线程的线程控制块（TCB）里，相当于“存档”。

2）然后调度器选出下一个要运行的线程。这个决策基于调度算法，比如 CFS（完全公平调度器）在 Linux 中就是通过红黑树管理可运行线程，取最左边节点。

3）接着恢复目标线程的上下文。把之前保存的寄存器、PC、栈指针等从它的 TCB 恢复到 CPU，CPU 就“续上”这个线程之前的工作。

整个过程不涉及虚拟内存空间切换（除非是进程切换），因为同一进程内的线程共享地址空间。但线程栈是独立的，所以栈指针必须换。

一次上下文切换通常消耗 2000~10000 个 CPU 周期，属于比较重的操作。高并发场景下频繁切换会拖累性能，像 Netty 这类框架就尽量用事件循环减少线程切换。



听说过 CFS 吗？（Linux）

CFS 全称是完全公平调度器（Completely Fair Scheduler），2007 年从 Linux 2.6.23 开始成为默认的进程调度器。它不按时间片轮转，而是盯住每个进程能分到的 **CPU 运行时间**，尽量让所有可运行进程“雨露均沾”。

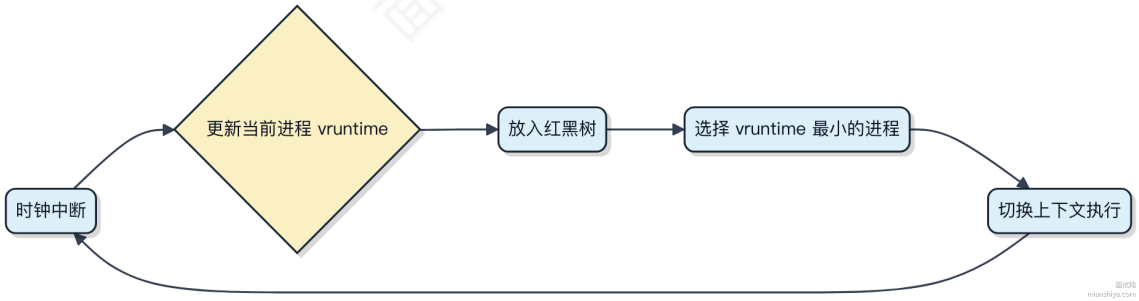
核心思路是用一个红黑树管理所有可运行进程，以虚拟运行时间 `vruntime` 为键。哪个进程的 vruntime 最小，说明它最近占 CPU 少，就该优先被调度。每次时钟中断都会更新当前进程的 vruntime，调度器选下一个任务时直接找树最左节点，O(log n) 时间搞定。

1）vruntime 计算会考虑进程权重，比如通过 `nice` 值调整。nice 值低的进程权重高，vruntime 增长得慢，就能多跑一会儿。

2）CFS 不给每个进程固定时间片，而是有个目标延迟机制，默认 6ms。假如系统有 4 个进程，那每个理想分 1.5ms。进程越多，分配的时间片越小，但尽量保证都在目标延迟内被调度一次。

3）休眠进程的 vruntime 不变，醒来后因为值偏低，会快速得到调度，有利于交互式程序响应。

别跟实时调度器搞混了，CFS 负责的是 SCHED_NORMAL 类型的任务。像数据库、Web 服务这类通用负载，基本都归它管。



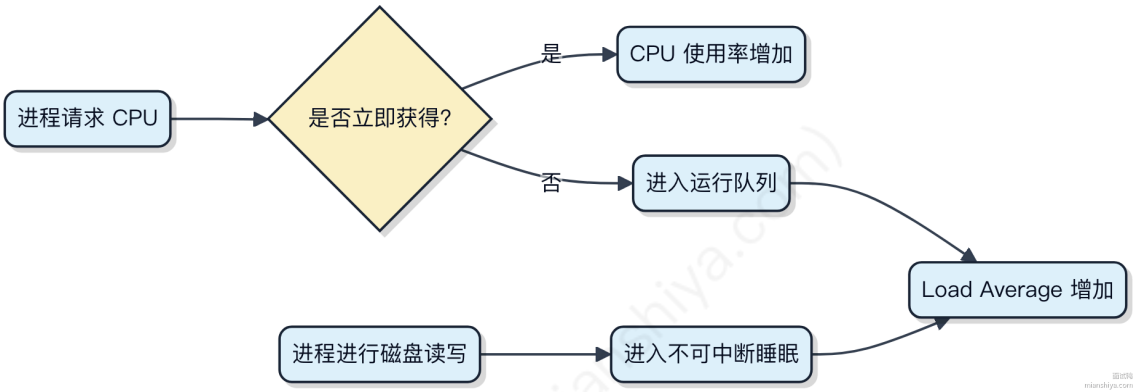
CPU 使用率和 CPU 负载指的是什么？它们之间有什么关系？

CPU 使用率反映的是 CPU 正在执行任务的时间占比，比如跑了 80% 的时间在干活，使用率就是 80%。它通常按用户态、系统态、等待 I/O 等维度细分。

CPU 负载（Load Average）则表示一段时间内，处于**可运行状态**和**不可中断状态**（如等待磁盘 I/O）的平均进程数。Linux 里一般显示 1 分钟、5 分钟、15 分钟三个值。举个例子，负载为 2.0 意味着平均有 2 个进程在排队或运行。

它们的关系容易被误解。使用率高，负载不一定高。比如单核 CPU 跑一个计算密集型任务，使用率 100%，但负载也就 1 左右。反过来，负载高使用率可能不高——大量进程在等 I/O，CPU 空闲，但负载上去了。

判断系统瓶颈得结合看。使用率接近 100% 说明 CPU 是瓶颈；负载远高于 CPU 核数（比如 4 核机器负载长期 > 8），说明系统整体压力大，可能是 I/O 或进程调度问题。



排查时 top 命令一眼就能看到这两个指标。如果负载高但使用率低，重点查 iowait 和 dstat 看磁盘；如果都高，考虑扩容或优化代码逻辑。

为什么要有虚拟内存？

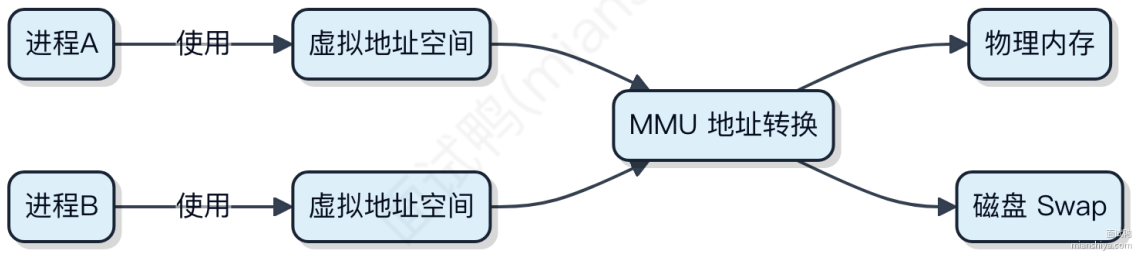
物理内存是有限的，比如一台机器只有 16GB RAM，但现代应用动不动就要用几十 GB 的地址空间。如果没有虚拟内存，程序直接操作物理地址，不仅会相互干扰，还容易因为内存不足直接崩溃。

虚拟内存的核心在于**地址隔离**和**内存扩展**。每个进程看到的都是独立的虚拟地址空间，从 0 开始编号，操作系统和 MMU（内存管理单元）负责把虚拟地址翻译成物理地址。这样两个进程可以有相同的虚拟地址，但实际映射到不同的物理内存上，互不干扰。

另一个关键是**分页机制**。系统把内存和磁盘划分为固定大小的页，一般是 4KB。当物理内存紧张时，操作系统可以把暂时不用的页换出到磁盘（swap），需要时再加载回来。虽然磁盘慢，但至少让程序能“假装”自己拥有大片内存。

这带来的好处很明显： 1）进程间内存隔离，一个崩了不影响别人 2）支持比物理内存更大的寻址空间 3）内存分配更灵活，可以按需映射

不过也有代价，比如缺页中断处理、TLB 命中率问题，都会影响性能。但整体来看，虚拟内存是现代操作系统能稳定运行多任务的基础设计。



进程的调度算法你知道吗？

进程调度算法其实是操作系统做任务分配的核心逻辑，关键在于怎么平衡响应时间和吞吐量。

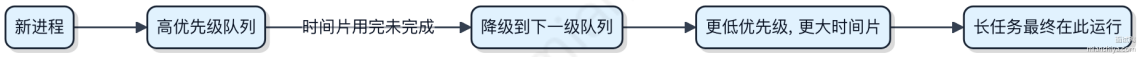
1）先来先服务（FCFS）就是排队等，谁先到就先跑。优点是简单直观，但短任务可能被长任务卡住，平均等待时间容易变差。

2）短作业优先（SJF）挑预计运行时间最短的先执行，能最小化平均等待时间。但实现难点在怎么预估时间，而且长任务可能一直被挤，出现“饥饿”。

3）时间片轮转（RR）给每个进程分一个固定时间片，轮流执行。比如 10ms 轮一次，保证所有任务都能及时响应，适合交互式系统。时间片太小切换开销大，太大又退化成 FCFS。

4）多级反馈队列（MLFQ）算是综合方案，把多个优先级队列串起来。新进程进高优先级队列，用完时间片没跑完就降级。低优先级队列的时间片更大，既能照顾响应速度，又能处理长任务。

Linux 实际用的是 **完全公平调度器**（CFS），它不靠静态优先级，而是跟踪每个进程的运行时间，总让运行时间最少的那个跑，相当于动态加权的轮转。



实时场景会用 **SCHED_FIFO** 或 **SCHED_RR**，比如音视频处理要求严格时序，就不能被普通任务打断。

什么是物理地址，什么是逻辑地址？

物理地址是内存芯片级别的真实地址，CPU 通过它直接访问内存硬件。你程序里看到的变量地址都不是这个，而是逻辑地址。

逻辑地址是操作系统给每个进程“画”的独立内存空间，比如 32 位系统就是从 0 到 4GB 的虚拟范围。每个进程都觉得自己独占内存，其实这是被系统管理过的假象。

真正干活的是 MMU（内存管理单元），它靠页表把逻辑地址翻译成物理地址。每次内存访问都要查页表，命中就转成物理地址去取数据，没命中就触发缺页中断，由操作系统从磁盘加载数据并更新页表。

这样设计的好处很明显：进程隔离、内存保护、支持比物理内存更大的寻址空间。像 Java 程序里的对象引用，全都是在逻辑地址空间里跑，压根不经过物理地址。

举个例子，两个进程都访问地址 0x1000，MMU 会根据各自的页表映射到不同的物理位置，互不影响。

页大小通常是 4KB，一个进程的逻辑页可以映射到任意物理页框，实现非连续内存分配，解决外部碎片问题。

Select、Poll、Epoll 之间有什么区别？

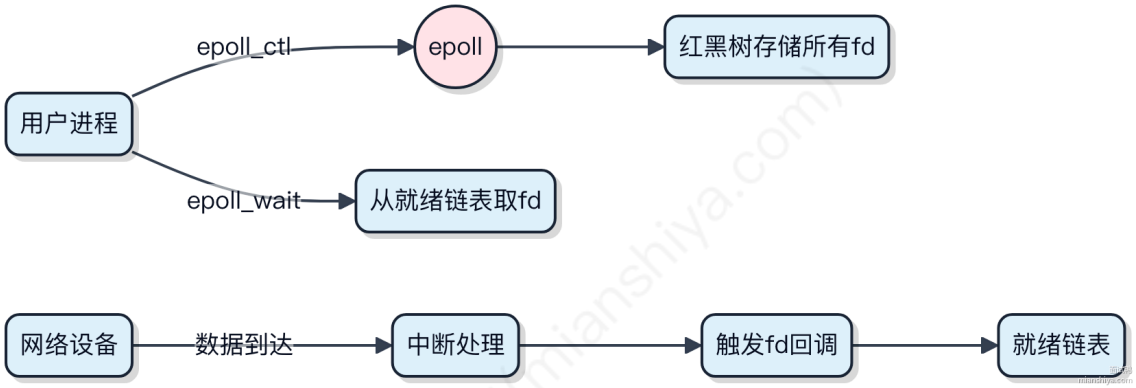
select、poll 和 epoll 都是 Linux 下处理 I/O 多路复用的机制，用来让一个线程能同时监控多个文件描述符的读写状态。它们的核心目标是一样的，但实现方式和性能差异很大。

1）select 使用固定大小的位图来表示 fd 集合，通常最大支持 1024 个 fd。每次调用都要把整个 fd 集合从用户态拷贝到内核态，返回后还要遍历所有 fd 判断是否就绪。这种方式在连接数多时开销大，效率低。

2）poll 其实跟 select 差不多，只是换成了链表结构传递 fd，突破了 1024 的限制。但它依然需要遍历所有 fd，而且每次都要传全量数据，**没有解决根本的性能问题**。

3）epoll 完全换了个思路，它在内核里用红黑树管理 fd，用就绪队列回调通知。调用 epoll_ctl 添加 fd 时注册回调函数，一旦有事件触发，内核直接把 fd 放进就绪队列。epoll_wait 调用时只返回就绪的 fd，不用遍历全部。这种机制让

单个线程轻松扛住几万并发，Nginx、Netty 都靠它撑着。



简单说，select 和 poll 是“你有没有事？我去挨个问一遍”，而 epoll 是“有事我主动告诉你”。连接少的时候差别不大，上几千并发时，epoll 的优势就压倒性了。

同步、异步、阻塞、非阻塞的I/O的区别？

同步和异步，重点在“谁通知结果”。同步是自己主动去问，结果得等服务端处理完才能拿到；异步是你发起调用后就不管了，系统处理完了会通过回调、事件告诉你。

阻塞和非阻塞，重点在“调用期间能不能干别的”。阻塞就像打电话点外卖，打完电话你就得拿着手机等，啥也干不了；非阻塞是你打完电话，该刷剧刷剧，外卖到了自然会喊你。

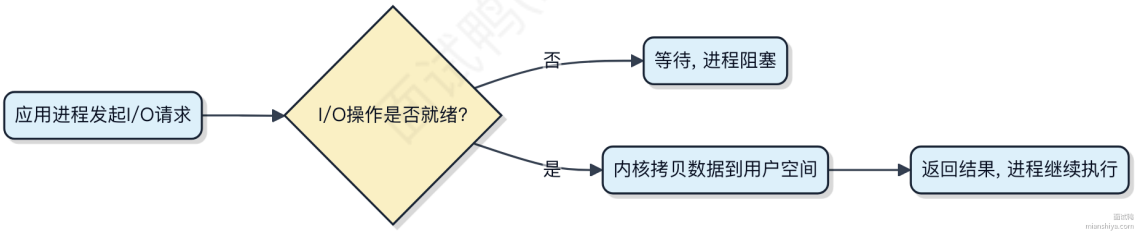
常见的组合是：

1）同步阻塞：发请求后线程挂起，直到数据返回。比如传统 `read()` 从网络读数据，没准备好就卡住。

2）同步非阻塞：轮询检查数据是否就绪，不挂起线程。比如设置 socket 为 non-blocking，反复 `recv()` 看有没有数据。

3）异步非阻塞：发起 I/O 操作后立即返回，内核完成读写后通知程序。比如 Linux 的 `epoll` 配合事件驱动，Node.js、Netty 就是这路子。

Java NIO 就是典型的异步非阻塞模型，用少量线程能扛住几万并发连接，脏活累活交给操作系统底层的多路复用器。



像 Redis 6.0 引入多线程 I/O，就是把网络读写（socket read/write）放到多个线程做，但命令执行还是单线程，本质上用的是**非阻塞多路复用 + 线程分工**。

阻塞和非阻塞的区别？

阻塞和非阻塞，说的其实是调用方在发起一个操作后，能不能干别的事。

1）阻塞调用，线程会停在原地等结果。比如读文件时数据还没到，线程就卡着不动，啥也干不了。像传统 `InputStream.read()` 就是这样，得等到有数据或者出错才返回。

2）非阻塞调用，调用立刻返回，不会让线程傻等。比如设置 Socket 为非阻塞模式后，`read()` 没数据就直接报“现在没数据”，程序可以去处理其他任务，过会再来看看。

关键在于**控制权是否交还给调用者**。阻塞模式下线程被操作系统挂起，浪费资源；非阻塞则把轮询的主动权交给应用自己掌握。

典型的例子是 NIO。`Selector` 配合非阻塞 Channel，一个线程能同时盯着成百上千个连接，哪个就绪了就处理哪个，撑起高并发的网络服务。

代码上差别也很明显：

```
// 阻塞：线程卡在这里
int data = inputStream.read();

// 非阻塞：立刻回来，靠返回值判断
int n = channel.read(buffer);
if (n == -1) handleClosed();
else if (n == 0) continue; // 没数据，继续轮询
```

真正高性能系统往往结合非阻塞 I/O 和事件驱动，比如 Netty 就是这套组合拳的集大成者。

同步和异步的区别？

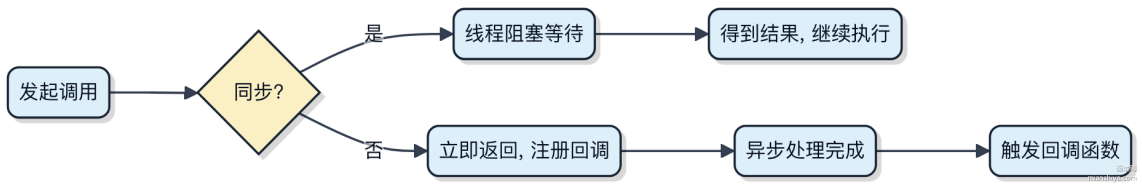
同步和异步的差别，关键看调用方要不要等结果。

1）同步操作里，线程发起一个调用后就卡着不动，得一直等到对方返回结果才能继续往下走。比如你去餐厅点餐，服务员站在你桌边等着厨房出菜，这期间他啥也不干，这就是典型的同步阻塞。Java 里的 `Thread.join()` 就是这样，主线程得等子线程执行完才能继续。

2）异步操作完全不一样，调用一发出，当前线程立马就能干别的事，不用傻等。等被调用方处理完了，会通过回调、事件通知或者 Future 的方式告诉你结果。像发个短信说“饭好了”，你收到再过去拿，中间时间全利用起来了。`CompletableFuture.supplyAsync()` 就是这种模式，任务扔进线程池就不管了，后面用 `.thenApply` 处理结果。

实际场景中，数据库连接池 HikariCP 在获取连接时可以选择同步等待，而像消息队列 RabbitMQ 的消费者则是异步接收消息，靠监听器触发处理逻辑。

系统吞吐量上，异步天然适合高并发场景。Tomcat 的 NIO 模式就是把请求转成事件注册到 selector，一个线程能扛住几千连接，压根不经过阻塞等待那套流程。



别把异步和多线程划等号，异步是一种编程模型，多线程只是实现手段之一。Node.js 单线程也能做异步 I/O，靠的是事件循环。

I/O模型有哪些？

I/O 模型主要就五种，面试常问的是阻塞、非阻塞、I/O 多路复用、信号驱动和异步 I/O。前四种都属于同步 I/O，只有最后一种是真正的异步。

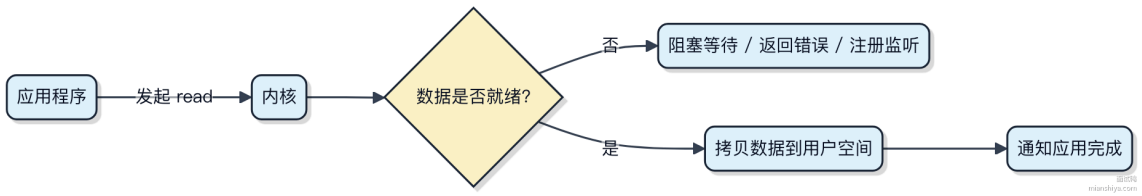
1）**阻塞 I/O** 是最简单的模型，比如调用 `read()` ，如果数据没到内核缓冲区，线程就一直卡着不动。Java 传统的 BIO 就是这种，一个连接一个线程，扛不住高并发。

2）**非阻塞 I/O** 在发起 read 调用后会立刻返回，通常返回 EAGAIN 或 EWOULDBLOCK 错误。应用要不断轮询去查数据到了没，CPU 浪费严重，实际很少直接用。

3）**I/O 多路复用** 是目前主流方案，一个线程能监控多个文件描述符。select、poll、epoll 都属于这类。像 Redis 就用 epoll，单线程处理上万连接没问题。epoll 支持边缘触发和水平触发，效率最高。

4）**信号驱动 I/O** 用 SIGIO 信号通知应用数据就绪，但信号处理机制复杂，容易出错，用得不多。

5）**异步 I/O**（AIO）是真异步，发起 read 请求后系统负责把数据从内核拷贝到用户空间，完成后再回调。Java 的 `AsynchronousSocketChannel` 就是这玩意，但 Linux 内核对 AIO 支持不友好，性能不如 epoll，Netty 也没选它。



简单说，**I/O 多路复用**是高性能网络编程的标配，AIO 理论最优但现实骨感。

为什么网络 I/O 会被阻塞？

网络 I/O 阻塞的根本原因在于 **内核空间与用户空间的数据未就绪**，这时候调用读写会卡住线程。

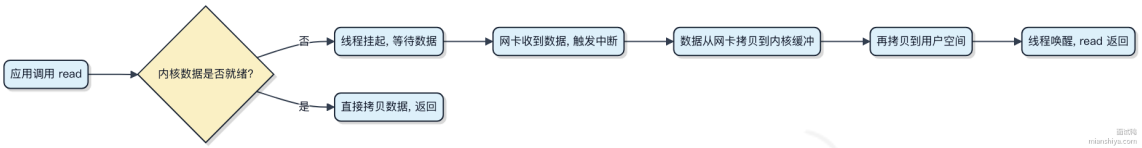
比如你用 `socket.getInputStream().read()` ，如果网卡还没收到数据，操作系统就会让线程休眠，直到有数据到达并拷贝到用户缓冲区。这个过程里线程啥也干不了，就是典型的阻塞。

1）数据没来：TCP 数据包还在路上，接收缓冲区空着，read 调用只能等。 2）数据来了但不全：比如你要读 1KB，只到了 300 字节，那剩下的还得继续等。 3）发送缓冲区满：发数据时如果网络慢，内核的发送队列满了，write 调用也会卡住。

这种模式在连接数少、请求稀疏时没问题，但像 Web 服务这种动不动几千并发的场景，每个线程占着内存和栈，压根扛不住。

```
// 阻塞式读取
ServerSocket server = new ServerSocket(8080);
Socket client = server.accept(); // 这里阻塞
InputStream in = client.getInputStream();
int data = in.read(); // 没数据就一直等
```

后来搞出 NIO，用 Selector 让单线程轮询多个连接，哪个就绪了才去处理，避免为每个连接开线程。Netty 就是基于这玩意做的高性能通信框架。



到底什么是 Reactor?

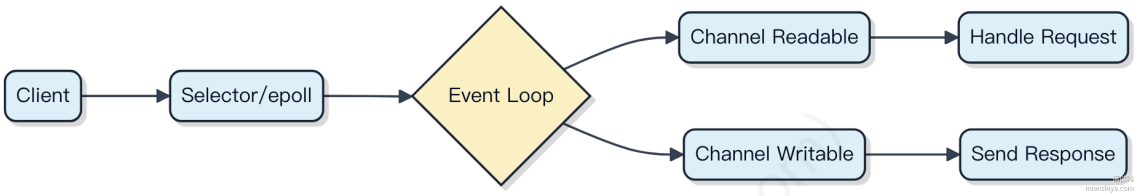
Reactor 不是某个具体框架，而是一种**事件驱动**的编程模型，用来构建高并发、低延迟的服务端应用。它用少量线程处理成千上万的连接，靠的是操作系统提供的 I/O 多路复用机制，比如 Linux 的 epoll。

整个模式的核心是一个或多个事件循环线程（Event Loop），每个线程监听一批通道（Channel）上的事件，比如数据可读、可写。一旦有事件就绪，就调用对应的处理器去处理，不会阻塞等待。

1）所有 I/O 操作都是非阻塞的
2）请求到来时不创建新线程，由 Event Loop 统一派发
3）业务逻辑以回调方式注册，事件触发时执行

像 Netty、Node.js、Spring WebFlux 都是 Reactor 模式的典型实现。拿 Netty 来说，Boss 线程负责 accept 新连接，Worker 线程用 Reactor 模式处理读写事件，一个 Worker 能扛住几万个连接。

代码层面你会看到 `ChannelHandler` 、 `onRead` 、 `onWrite` 这类回调方法，而不是传统的同步方法调用。这种风格一开始不太适应，但吞吐量上来了，线程开销也压到了最低。



注意别把 Reactor 和响应式编程混为一谈。Reactor 是底层线程模型，响应式（如 Reactor 库）是基于它的编程抽象，虽然名字撞了，但层次不同。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*